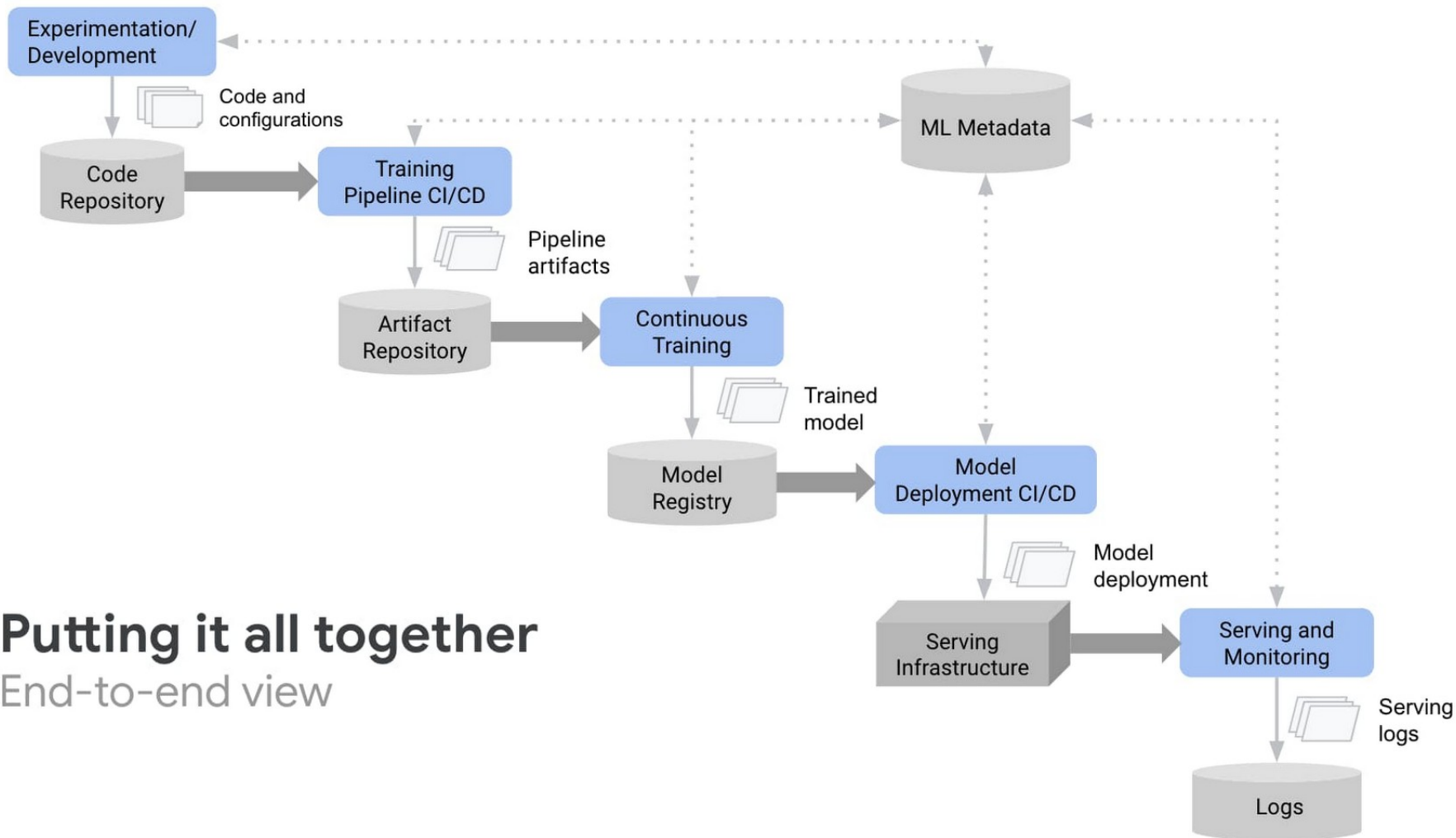


A large, dark blue ink splatter or blotch is centered on a white background. The splatter has irregular, feathered edges with some smaller droplets and speckles trailing off from the main mass. The text is centered within the dark blue area.

AI Infrastructure

Ioannis Papapanagiotou

ML Feature Stores



Putting it all together

End-to-end view

ML Stores

- The typical automated model pipeline in enterprise production environments include 3 types of stores, such as
 - **feature store**
 - **metadata store**
 - **model registry**

Feature store

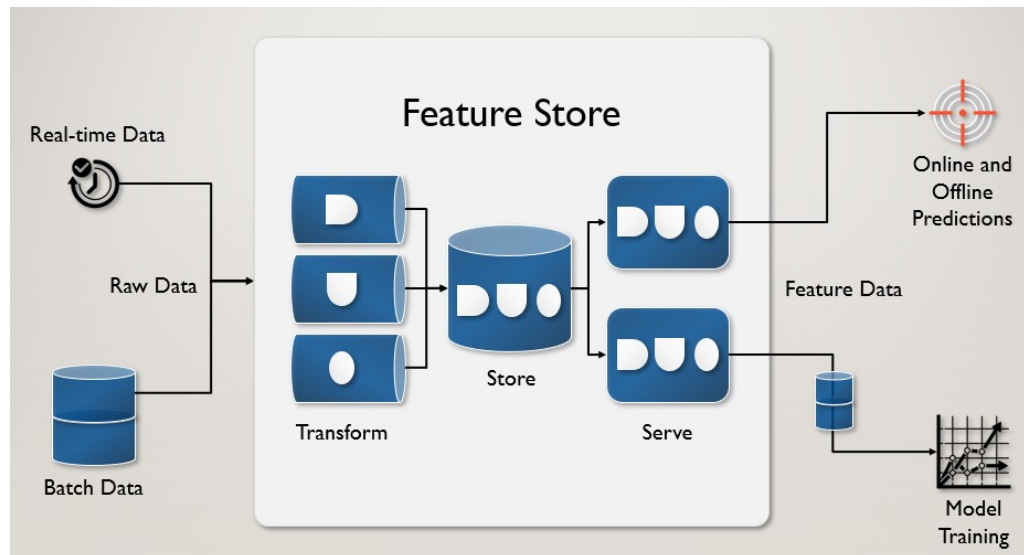
- A feature store is an emerging data system used for machine learning, serving as a centralized hub for storing, processing, and accessing commonly used features.
- It's making them available for reuse in the development of future machine learning models.
- Feature stores operationalize the input, tracking, and governance of the data as part of feature engineering for machine learning.

Feature Store use-cases

- To fully understand why feature stores are so important, one needs to have a basic understanding of how machine learning models work.
- ML models use features, a measurable piece of data that can be used to teach the model to make predictions about the future based on data from the past.
- For example, to predict whether a customer will make a purchase within the next month, variables or features such as the sum of last month's purchases or the number of website visits this week can be used.
- Similarly, for a medical-related use case, features used to describe a medical patient may include variables such as age, weight, tobacco use, exercise frequency, and current medical diagnosis.

Feature Store

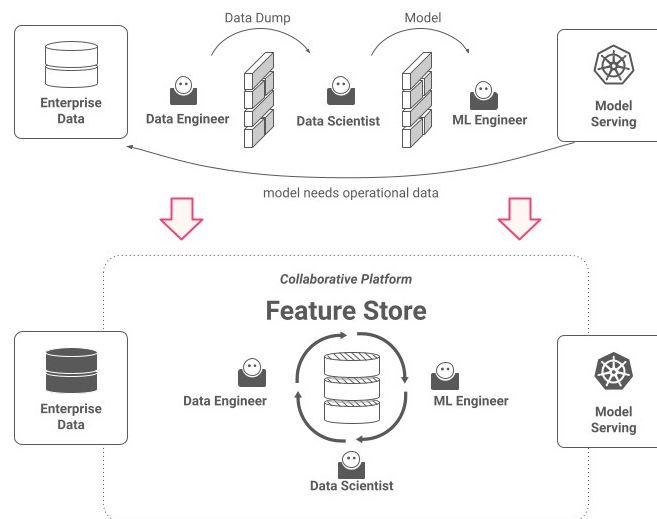
- contains data extracted from various source systems and transformed into the features as required by the model.
- The ML pipeline takes the data in batches from the feature store to train the model.



Benefits of Feature Store

Enable feature reuse

- Once features have been developed, they can be saved in the feature store. This makes them available to be reused or shared between ML models and teams. Developing new features is time-intensive, keeping data scientists locked into tasks that could have been completed more efficiently by repurposing an existing feature. A well-stocked feature store can be accessed to quickly create new ML models by eliminating the need to build each new feature from scratch.



Benefits of Feature Store

Ensure feature consistency

- Understanding how a feature was developed, how it was computed, and what information it represents is important.
- Maintaining consistent definitions and development documentation can be a challenge, especially for larger organizations.
- A centralized feature store solves this, providing a single registry for all ML features that's easily accessible to all teams within the business.

Benefits of Feature Store

Maintain peak model performance

- When there is a discrepancy between how features are defined for training and how they are implemented in serving pipelines, it can lead to reduced performance of models in production
- . And because production data will evolve over time, monitoring the profile of the data set over time is important to maintain the highest model performance.
- To solve this problem, feature stores have centralized feature pipelines that ensure feature definitions and their implementation remain consistent across training and inference and include continuous monitoring of data pipelines.

Benefits of Feature Store

Enhance security and data governance

- Quickly identifying what data a model was trained on and what data it was fed after deployment is important for iterating or debugging.
- A feature store contains detailed information for each machine learning model, such as what data was used on it and when.
- Feature stores that integrate into a cloud data warehouse benefit from enhanced data security that comes with this configuration, providing additional security for both the models and the data they were trained on.

Benefits of Feature Store

Foster collaboration between teams

- A feature store offers a centralized platform for the development, storage, modification, and reuse of ML features. This fosters cross-team collaboration, allowing members from multiple data science teams the ability to share ideas, and develop and track the progress of features that may be useful for multiple business applications.

Metadata Store

metadata store is a centralized model tracking (bookkeeping) system, maintained at the enterprise level, contains the model metadata at each stage of pipeline.

The model metadata store facilitates the model stage transition, say from staging to production to archived.

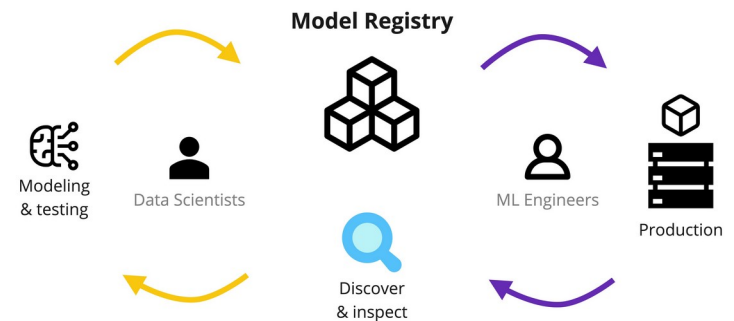
The model training is performed in one environment and deployment in other environments where the model inference will be performed just by specifying the remote model file path.

The model metadata store is used for model experiments tracking and compare model experiments w.r.t. its performance.

The model metadata includes training data set version, links to training runs and experiments.

Model Registry

- contains data of all trained models such as trained model version, model training date, model training metrics, hyperparameters used for training the model, predicted outcomes, and diagnostic charts (confusion matrix, ROC curves).
- The appropriate model will be picked from the model registry based on the intended target user's requirement.
- The model metadata store & model registry can be implemented (custom) using a light-weight database, such as SQLite and a set of python functions; or a set of open-source/proprietary products, such as MLflow (community edition/managed service from Databricks). This tool provides an ability to manage models using a GUI or a set of APIs.

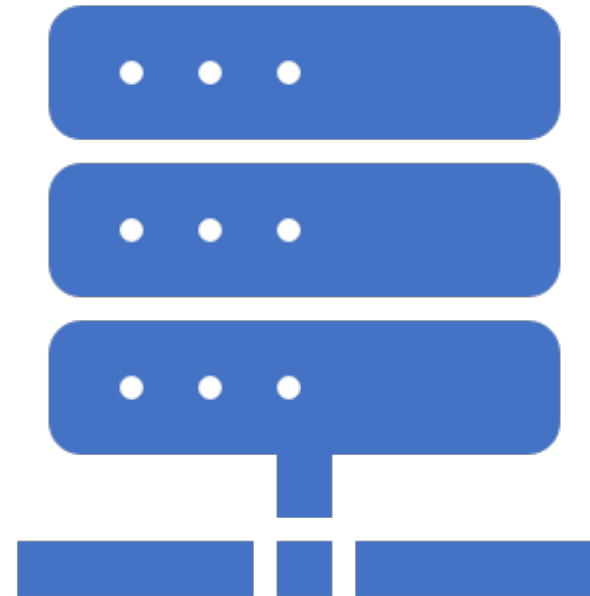


Model Deployment

- The models can be deployed in production environments to perform prediction either in batch inference mode or on-line inference mode.
- The batch inference can be achieved by scheduling as a job to run at a time interval and send the results via email to the intended users.
- The on-line inference can be achieved by exposing the model as a web service using frameworks such as python flask library or Streamlit library to develop interactive web applications and invoke the model using its HTTP endpoint.

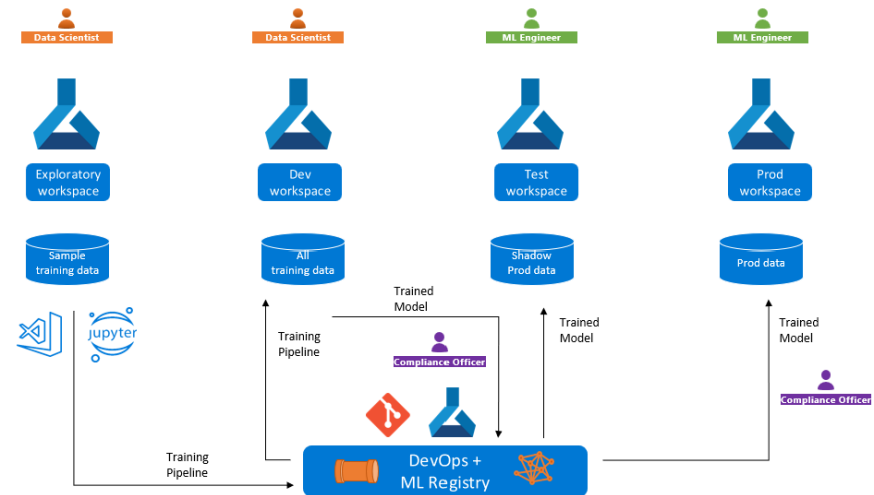
Cross-workspace MLOps with registries

- A registry, much like a Git repository, decouples machine learning assets from workspaces and hosts the assets in a central location, making them available to all workspaces in your organization.
- You can use registries to store and share assets such as **models, environments, components**, and **datasets**.
- To promote models across development, test, and production environments, you can start by iteratively developing a model in the development environment.
- When you have a good candidate model, you can publish it to a registry.
- You can then deploy the model from the registry to endpoints in different workspaces.



Cross-workspace MLOps with registries

- To develop a pipeline in one workspace and then run it in other workspaces, start by registering the components and environments that form the building blocks of the pipeline. When you submit the pipeline job, the compute and the training data, which are unique to each workspace, determine the workspace to run in.
- The following diagram shows training pipeline promotion between exploratory and development workspaces, then trained model promotion to test and production.



Model Packaging/Distribution

- The trained ML model is serialized into various formats to be able to distribute the model for deployment into TEST/PROD environments.
- The most common formats are pickle (for machine learning or deep learning models developed in python), ONNX (Open Neural Network Exchange format, for deep learning models), and PMML (Predictive Model Markup Language XML-based format, specifically for models developed using logistic regression and neural networks).
- The model can be packaged as a .jar file or as a docker container image and deployed as a prediction service into production.

Model Packaging/Distribution

- The trained ML model is serialized into various formats to be able to distribute the model for deployment into TEST/PROD environments.
- The most common formats are pickle (for machine learning or deep learning models developed in python), ONNX (Open Neural Network Exchange format, for deep learning models), and PMML(Predictive Model Markup Language XML-based format, specifically for models developed using logistic regression and neural networks).
- The model can be packaged as a .jar file or as a docker container image and deployed as a prediction service into production.

Model Containerization

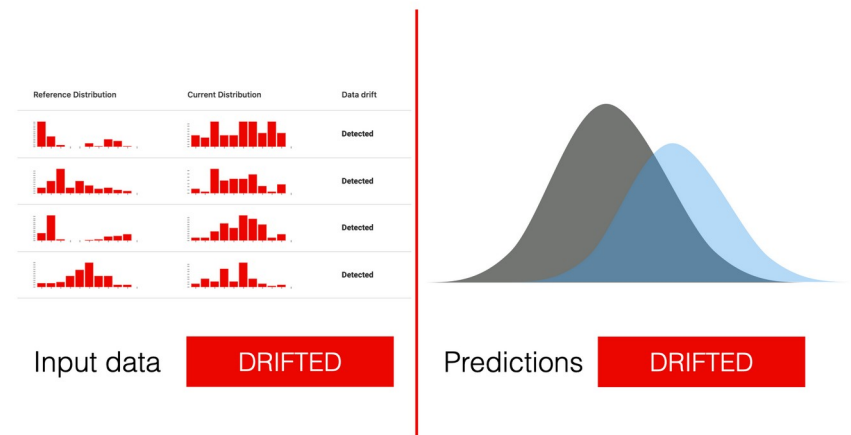
- can be achieved by building a docker image, bundled with training and inference code, along with the necessary training and testing data and the model file for future predictions.
- Once the docker file is created bundled with necessary ML model, a CI/CD pipeline can be built using a tool, such as Jenkins. This docker container image can be exposed as a REST API, so that any external stakeholders can consume this ML model, either from on-premises or public cloud (in case of high compute requirements for building a deep learning model).
- In case of packaging and managing multiple docker containers, **Kubeflow**, the ML toolkit for Kubernetes can be used.

Model Performance Monitoring

- is an important activity where the predicted outcome (e.g., predicted sale price of an item) vs. actual value (actual sale price) is continuously monitored.
- And it is recommended to understand the end users' reaction to the final predictions. In some scenarios, it is recommended to keep old model and new model running in-parallel to understand the variation in performance in both the models (model validation).

Data Drift

- It is mandatory to address performance degradation that arise due to 'data drift' (underlying source data pattern changes due to seasonality and trend, e.g., year-end sales data (or) addition of new categories (or) one attribute is not being generated by the upstream systems) and 'model drift' (statistical properties of target variable changes, e.g., definition of a fraudulent transaction itself changes).
- The most accurate way to measure the model drift is by measuring the F1 Score that combines the precision and the recall of a classifier into a single metric by taking their harmonic mean.
- The model will be retrained as an when the model drift (F1 Score) falls below certain threshold or at regular intervals (batch mode) or train the model as soon as the data is available (online training). It is very important to collect model logs and prediction logs by using popular logging tools such as elasticstack, and fluentd.

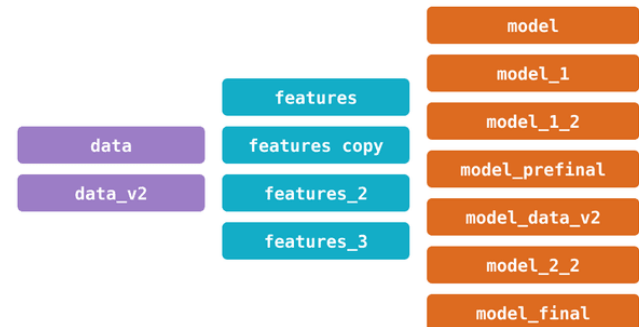


Model Version Management

- Mandatory activity in ML model management as the model will be retrained at regular intervals due to changes in the underlying source data or as demanded by audit and compliance purposes.
- The source data, the model training scripts, model experiment, and the trained model are versioned together in the code repository.
- There are open-source tools available, such as Data Version Control (DVC) or AWS CodeCommit that uses Git as underlying code repository for model version management purposes.

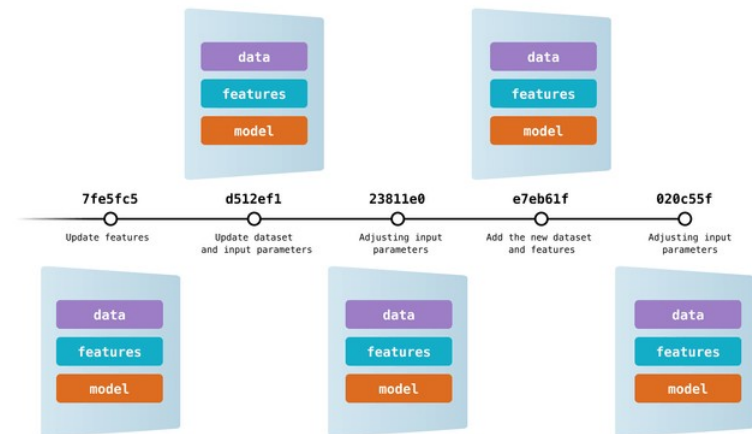
Versioning Data and Models

- Data science teams face data management questions around versions of data and machine learning models. How do we keep track of changes in data, source code, and ML models together? What's the best way to organize and store variations of these files and directories?
- Another problem in the field has to do with bookkeeping: being able to identify past data inputs and processes to understand their results, for knowledge sharing, or for debugging.



Data Version Control

- **Data Version Control (DVC)** lets you capture the versions of your data and models in [Git commits](#), while storing them on-premises or in cloud storage. It also provides a mechanism to switch between these different data contents. The result is a single history for data, code, and ML models that you can traverse — a proper journal of your work!
- DVC enables data *versioning through codification*. You produce simple [metafiles](#) once, describing what datasets, ML artifacts, etc. to track. This metadata can be put in Git in lieu of large files.
- You can use DVC to create [snapshots](#) of the data, [restore](#) previous versions, [reproduce](#) experiments, record evolving [metrics](#), and more!



ML Model Deployment Best Practices

The recommended model deployment best practices are:

1. Automate the steps required for ML model development and deployment, by leveraging DevOps tools that gives some extra time for model retraining;
2. Perform continuous model testing, performance monitoring and retraining after its production deployment to keep the model relevant/current as the source data changes to predict the desired outcome(s);
3. Implement logging while exposing ML models as APIs, that includes capturing input features/model output (to track model drift), application context (for debugging production errors), model version (if multiple re-trained models are deployed in production);
4. Manage all the model metadata in a single repository.

References

- [What is a Feature Store in Machine Learning? \(Snowflake Guides\)](#)
- [Machine Learning Registries for MLOps \(Microsoft\)](#)